Nevertheless, to give you an intuition of why it works this way, consider a two-dimensional space: one point divides it into two halves, two points in three regions, 3 points will create four regions etc..., in general n points will create n+1 regions; if the tree is balanced and n is sufficiently big, we can assume these regions are approximately equally-sized.

Now suppose the dataset covers a unitary area[12], when we run a nearest neighbor-search we first traverse the tree from the root to the closest leaf[13], and this, for a balanced tree, means traversing `O(log(n))` nodes. Since we hypothesized that each region is approximately of the same size, and we have n+1 of them, the area covered by the closest leaf will also be a rectangular region of area approximately equal to `1/n`. That means that there is a reasonably high probability that the distance between the target point and the nearest neighbor we have found during this traversal is no larger than half the diagonal of the region, which in turn is smaller than the square root of the region's area, or in other words, .



